

Locale by ACHIEVEMOR: NotebookLM Index Pages

Complete knowledge base structure for building the voice-first AI business orchestration platform

This document provides six concrete NotebookLM "index pages" - each one is a checklist of PDFs, web docs, and internal specs to load for a specific technical domain in the Locale architecture[1][2][3].

Index 1: Hostinger + Firebase + GCP Core Architecture

Purpose: Design the hybrid stack where Hostinger VPS runs Locale's Next.js frontend, n8n automation, and Dockerized agents, while Firebase/GCP provide auth, database, and AI services[1][3].

Internal Documents to Upload

- **Locale-UI-and-Infrastructure-building.txt** – Defines Next.js 14 App Router, Firebase integration, Luc usage calculator, voice modules, and workspace layout[1]
- **Locale by achvmr.pdf** (both versions) – High-level product vision, architecture diagrams, and system overview[2]
- **Locale-projectsummary.pdf** – Overall system summary and key technical decisions[2]
- **comprehensiveintegrationguide.pdf** – Integration patterns and service connections[2]
- **Hostinger Strategy + Multi-Tenant SaaS Security note** – Details Hostinger cloud startup plan, KVM VPS specs (2+ vCPU, 8GB RAM), Docker support, firewall configuration, and tenant isolation patterns[3]
- **Current README.md** – Quick start guide, environment setup, and deployment instructions[1]

External Web Documentation to Add

- Hostinger KVM VPS documentation
 - Features, specifications, and pricing tiers
 - Docker and Docker Compose support
 - Resource limits (CPU, RAM, disk I/O)
 - Firewall configuration and security best practices
 - Snapshot and backup procedures
 - SSH access and deployment workflows
- Firebase documentation
 - Authentication (email/password, social providers, custom claims)
 - Firestore (data modeling, queries, security rules)
 - Cloud Storage (file uploads, security rules)
 - Cloud Functions (serverless triggers, HTTP endpoints)

- Firebase Analytics and monitoring
- Google Cloud Platform
 - Cloud Run (containerized services, auto-scaling)
 - Cloud Tasks (asynchronous job queues)
 - Cloud SQL (Postgres + pgvector for embeddings, Row Level Security)
 - Load Balancing and SSL certificate management
 - IAM and service account best practices
- Vertex AI documentation
 - Gemini models (Flash, Pro, Ultra)
 - Function Gemma (function calling and tool use)
 - RAG Engine (retrieval augmented generation)
 - Interactions API (multi-turn conversations)
 - Pricing and quota management
- n8n self-hosting documentation
 - Installation on VPS (Docker deployment)
 - Securing n8n with authentication and HTTPS
 - Webhook configuration and external triggers
 - Integration with Firebase, GCP, and APIs
 - Backup and version control for workflows
 - Resource requirements and scaling considerations

Key Research Questions for This Notebook

1. How to split responsibilities cleanly between Hostinger VPS (web app, n8n, Docker agents) and Firebase/GCP (auth, data, AI)[1][3]
2. Best deployment pattern for Next.js 14 App Router on Hostinger (Node/PM2 vs Docker) while securely calling Firebase and Vertex AI[1][3]
3. How to run n8n as a Docker container on Hostinger VPS alongside agent containers, with proper networking and security isolation[3]
4. How to model tenants (Clients vs Partners) in Firestore with security rules mirroring the Deome dual-gateway pattern[1][3]
5. How to log per-tenant usage across Hostinger containers, n8n workflows, and GCP services for Luc's cost calculator and billing[1][3]
6. Resource allocation strategy on Hostinger VPS: how much CPU/RAM for Next.js, n8n, Agent Zero, and OpenClaw containers[3]
7. Backup and disaster recovery patterns for VPS + Firebase + n8n workflows[3]

Prompts to Ask NotebookLM

- "Summarize how the current Locale UI/infrastructure doc expects Next.js and Firebase to interact; list concrete endpoints and services"[1]
 - "From all sources, sketch three candidate deployment topologies for Hostinger VPS + n8n + Firebase + Vertex AI, with pros/cons"[1][3]
 - "Extract every mention of 'multi-tenant' or 'tenant isolation' and propose a unified data model for Locale with Firestore security rules"[1][3]
 - "Generate a resource allocation plan for a Hostinger KVM 2 VPS (2 vCPU, 8GB RAM) running Next.js, n8n, Agent Zero, and OpenClaw containers"[3]
 - "Design a backup strategy covering VPS snapshots, Firebase exports, and n8n workflow versioning"[3]
-

Index 2: AI Agent Layer on Hostinger (ACHEEVY, Agent Zero, OpenClaw, n8n)

Purpose: Define how ACHEEVY orchestrates agents (Agent Zero for HBA, OpenClaw for Boomer_Angs, n8n for automation) running as Docker containers on Hostinger, and how those map to Locale's SaaS tiers[1][2][3].

Internal Documents to Upload

- **Locale-UI-and-Infrastructure-building.txt** – Includes ACHEEVY as Digital CEO, BoomerAng personas, CircuitBox workspace, and agent-to-workspace attachment logic[1]
- **Locale The Autonomous White-Label PMO Architecture** – Describes orchestrator patterns, Gemini integration, Function Gemma, LangGraph, agent teams, and cost tracking[1]
- **BUILD DIRECTIVE: THE AUTONOMOUS MEDIA PIPELINE** – Defines C1 Thesis, Nano Banana avatars, media orchestrator, Cloud Tasks, and synthetic SME pipeline[2]
- **BUILD DIRECTIVE: THE SME SWIPE PLAYGROUND VOICE STACK** – Maps SME personas, voice stack integration, and explore/swipe experience to concrete components[2]
- **Hostinger strategy + multi-tenant SaaS security note** – Includes Docker isolation patterns, tier-based routing, and container resource limits[3]
- **Current README.md** – Documents Agent Zero and OpenClaw Docker deployment instructions[1]

External Web Documentation to Add

- Agent Zero documentation
 - GitHub README and architecture overview
 - Installation and deployment guides
 - API endpoints and authentication
 - Memory and context management
 - Tool integration patterns
 - Security best practices
- OpenClaw documentation
 - GitHub repository and setup instructions
 - Gateway architecture and device pairing
 - TLS configuration and security model
 - API specifications
 - Multi-agent coordination patterns
- LangGraph or equivalent multi-agent frameworks
 - Multi-agent orchestration patterns
 - State management across agents
 - Graph-based workflow design
 - Error handling and recovery
- n8n workflow automation
 - Creating custom nodes and integrations
 - Connecting to Agent Zero and OpenClaw APIs
 - Webhook triggers and HTTP requests

- Error handling and retry logic
- Workflow templates for common agent orchestration patterns
- Monitoring and logging n8n executions
- Docker Compose best practices
 - Multi-container networking
 - Volume management for persistent data
 - Resource constraints (CPU, memory limits)
 - Health checks and restart policies
 - Environment variable management

Key Research Questions for This Notebook

1. How ACHEEVY should expose a stable API on Hostinger that Locale (and later third parties) can call, independent of which agents run behind it[1][3]
2. Concrete Docker Compose layout on Hostinger VPS: which containers (Next.js, n8n, ACHEEVY, Agent Zero, OpenClaw, media orchestrator) and how to configure networking between them[3]
3. How to limit CPU/RAM and network bandwidth per container to prevent resource contention on shared VPS[3]
4. Mapping Free / Pro / Enterprise tiers to agent availability: which agents are shared vs get dedicated containers[3]
5. How n8n workflows can orchestrate Agent Zero and OpenClaw, triggering them based on user actions or scheduled tasks[1][3]
6. How to keep SmelterOS / Intelligent Internet style tooling as optional integration packs, not mandatory dependencies for Locale[1]
7. Security model: how agents access Firebase, GCP APIs, and user data without exposing credentials[3]
8. How to version and deploy agent updates without downtime on production VPS[3]

Prompts to Ask NotebookLM

- "From all docs, list every agent persona (ACHEEVY, BoomerAng, BoomerComms, HBA, etc.) and the concrete capabilities/tools they wrap"[1]
- "Generate a proposed Docker Compose file for Hostinger that matches the architecture described in the PMO/agent docs, including n8n"[1][3]
- "Explain how to refactor the older SmelterOS / House of ANG patterns into a lightweight agent layer for Locale only"[1]
- "Design an n8n workflow that orchestrates Agent Zero and OpenClaw to handle a business architecture request from a user"[1][3]
- "Propose a security model for agents running in Docker containers to access Firebase and GCP APIs using service accounts"[3]

Index 3: Voice-First Stack (Groq Whisper STT + Inworld/ElevenLabs TTS + Gemini)

Purpose: Nail the speech-to-text and text-to-speech stack for a voice-first PWA that feels like a real companion, with Groq Whisper as primary STT and premium TTS via Inworld AI (primary) + ElevenLabs (content creation)[1][2].

Internal Documents to Upload

- **STRATEGIC RECOMMENDATION: THE HYBRID COMPANION ENGINE** – Cost and latency analysis Groq vs OpenAI Whisper vs Deepgram vs Gemini, plus PWA voice companion design[2]
- **CORRECTED Copilot Instructions for Locale by ACHIEVEMOR – Voice Integration** – STT, TTS, Luc integration, VoiceWidget component, lib/voice utilities[1]
- **Locale-UI-and-Infrastructure-building.txt** – Includes VoiceWidget.tsx, lib/voice modules, voice API routes, and Luc's voice-driven UX[1]
- **BUILD DIRECTIVE: THE SME SWIPE PLAYGROUND VOICE STACK** – Defines voice provider switch (Inworld, ElevenLabs, Deepgram, Gemini), persona-specific voices, and Super Admin controls[2]
- **Current README.md** – Documents Groq Whisper and Inworld/ElevenLabs implementation examples[1]

External Web Documentation to Add

- Groq platform documentation
 - Whisper Large v3 and v3 Turbo API reference
 - Pricing and rate limits
 - Supported audio formats and file size limits
 - Language support (99 languages)
 - Latency benchmarks and speed factor measurements
 - Best practices for real-time transcription
- Inworld AI TTS documentation
 - TTS Max and TTS Mini model specifications
 - API reference and authentication
 - Voice catalog and free voice cloning
 - Latency characteristics (<250ms Max, <130ms Mini)
 - Pricing (\$10/M chars Max, \$5/M chars Mini)
 - 15-language support and pronunciation guides
- ElevenLabs documentation
 - Flash v2.5 and Multilingual v2 model specs
 - API endpoints and SDKs
 - Voice library and custom voice cloning
 - Voice settings (stability, similarity boost, style)
 - Pricing tiers and usage limits
 - 32-language (Flash) and 70+ language (Multilingual) support
- Deepgram documentation
 - Nova-2 model specifications
 - Streaming vs batch transcription
 - Pricing and performance benchmarks
 - Use cases as fallback or alternative to Groq
- Gemini documentation
 - Gemini Flash Audio capabilities
 - Gemini Voice API (if available)
 - Native audio understanding and generation
 - Pricing and latency characteristics
- PWA best practices
 - Audio recording in browser (MediaRecorder API)

- Microphone permissions and user consent
- Web Audio API for playback and processing
- Offline audio caching with Service Workers
- Manifest configuration for voice-first apps
- Push notifications for voice alerts

Key Research Questions for This Notebook

1. Exact routing logic for hybrid STT strategy: when to use Groq Whisper vs Deepgram vs Gemini audio, based on duration, user tier, or cost constraints[2]
2. How to implement the Super Admin "voice provider switch" and persona-specific voice IDs in Firestore schema and backend code[2]
3. UX patterns for a floating voice widget in a PWA that works from mobile browser home-screen shortcut[1][2]
4. How to combine STT transcription with an "active listener" LLM prompt that validates emotion, intent, and structures next actions[2]
5. Latency optimization: measuring end-to-end latency from mic input to audio playback and identifying bottlenecks[1][2]
6. Cost analysis: per-user monthly costs for different usage patterns (light, moderate, heavy voice use)[2]
7. How to cache frequently used TTS audio clips to reduce API calls and improve response time[1]
8. Integration with n8n: triggering voice workflows or processing transcripts through n8n automation[3]

Prompts to Ask NotebookLM

- "Using the hybrid companion engine doc, derive a concrete STT/TTS routing spec for Locale with per-tier settings and fallbacks"[2]
- "Summarize all implementation hints for VoiceWidget.tsx and lib/voice/* from the Copilot instructions and UI/infrastructure doc"[1]
- "Compare cost/latency of Groq Whisper vs Deepgram vs Gemini over 1, 10, and 100 hours of audio and recommend thresholds"[2]
- "Design a caching strategy for TTS audio to minimize API calls for common phrases and responses"[1]
- "Propose an n8n workflow that processes voice transcripts, extracts intent, and triggers appropriate agent actions"[1][3]

Index 4: Gateway, Verticals, SME Personas & Media Pipeline

Purpose: Design the dual Deome-style gateway (Client vs Partner), Verticals marketplace page, SME swipe playground, and synthetic media factory (video/audio/text from Nano Banana + Key AI + Google Vids)[1][2].

Internal Documents to Upload

- **Deom-Gateway-System.pdf** – Structural/security blueprint for the dual-gateway auth and routing system[1]
- **Master Architectural Prompt sections** – Gateway Infrastructure, Verticals Marketplace Engine, Plugs AI Utility Engine, Design System, Technical Stack Rules[1]
- **BUILD DIRECTIVE: THE SME SWIPE PLAYGROUND VOICE STACK** – SME swipe playground UX, persona vault schema, voice behavior, and discovery patterns[2]
- **BUILD DIRECTIVE: THE AUTONOMOUS MEDIA PIPELINE** – Hot tasks grid, C1 Thesis SME cards, Nano Banana avatars, Key AI / Google Vids workflow, Cloud Tasks orchestration[2]
- **Nothing-inspired aesthetic references** – Visual design mockups and Nerds Code dashboards used as design references[1]

External Web Documentation to Add

- Google Maps Platform documentation
 - Maps JavaScript API
 - Places API (business data, reviews, photos)
 - Tiles API (custom map styling)
 - 3D Maps and Photorealistic 3D Tiles
 - Pricing and usage quotas
 - Use cases for real estate and locality-aware features
- Key AI video generation documentation
 - API endpoints and authentication
 - Video creation from text/scripts
 - Customization options (avatars, backgrounds, voice)
 - Pricing and rendering time
 - Export formats and quality settings
- Google Vids documentation
 - API access (if available)
 - Video creation and editing capabilities
 - Integration with Google Workspace
 - Export and sharing options
- Swipe card UX best practices
 - Tinder-like swipe gesture implementation
 - Card stack animations and physics
 - Mobile touch gesture handling
 - Undo/redo patterns
 - State management for swiped cards
- Discovery pattern design articles
 - Recommendation algorithms
 - User preference learning
 - Content filtering and moderation
 - A/B testing for discovery UX
- n8n media pipeline workflows
 - Triggering video generation from scripts
 - Chaining Cloud Tasks for multi-step media creation
 - Error handling and retry logic for long-running media jobs
 - Webhook notifications when media is ready

Key Research Questions for This Notebook

1. Exact user flow from public homepage → Gateway A (Client) vs Gateway B (Partner) → Verticals → Hot Tasks and SME personas[1][2]
2. Firestore data model for Verticals, Hot Tasks, SME personas, and user preferences (saved, followed, engaged)[2]
3. How to turn "plugins" and GCP tools (Interactions API, Function Gemma, Codey, A2P, Maps, Firecrawl-style scrapers) into selectable AI utilities in the Verticals/AI tab[1][2]
4. How to build the media hand-off pipeline (script → audio → video) entirely on Locale infrastructure using Cloud Tasks and Cloud Run, with no Zapier/Make dependency[2]
5. Using n8n to orchestrate the multi-step media pipeline: script generation, audio synthesis, video creation, and final delivery[2][3]
6. How to store and serve generated media (videos, audio, images) efficiently on Firebase Storage or Cloud Storage[1][2]
7. How to implement the SME swipe playground with smooth animations and state management[2]
8. Security model: ensuring Clients only see approved Partners, and Partners control their visibility in Verticals[1]

Prompts to Ask NotebookLM

- "From all gateway and Deome docs, produce a detailed flowchart of auth, roles, and navigation for Clients vs Partners"[1]
- "Synthesize a single Firestore schema for sme personas, hotTasks, verticals, and userSavedSMEs that matches both the swipe playground and media pipeline directives"[2]
- "List every GCP or third-party API that appears in the Verticals / AI tab / media pipeline docs and group them by function"[1][2]
- "Design an n8n workflow that orchestrates the entire media pipeline from script input to final video delivery"[2][3]
- "Propose a content moderation strategy for user-generated SME personas and Partner profiles in the Verticals marketplace"[1][2]

Index 5: Data, Auth, Stripe Connect, Multi-Tenancy & Usage (Luc)

Purpose: Define auth flows for Clients vs Partners vs Admin, Stripe Connect with escrow, multi-tenant security, and Luc's usage/cost logic for metering and billing AI usage correctly[1][2][3].

Internal Documents to Upload

- **Gateway / Stripe / Locale Gateway System / Stripe Paywall Integration sections** – From Copilot instructions, describe Client/Partner split, Stripe Connect escrow, and tiered access[1]
- **Locale The Autonomous White-Label PMO Architecture – Infrastructure/Tenant Isolation/Billing Agent** – Cloud SQL + pgvector, Row Level Security, tenant-labeled usage, and nightly billing agent[1]
- **Multi-tenant SaaS security note in Hostinger strategy doc** – Firestore rules, Docker isolation, rate-limiting, and maximum-value plan strategy[3]

- **Luc Locale Usage Calculator instructions** – Luc's role, UI, Firestore tier integration, and how voice interfaces with cost estimation[1]
- **Current README.md** – Security best practices section and Firebase Analytics implementation[1]

External Web Documentation to Add

- Stripe Connect documentation
 - Marketplace and platform patterns
 - Destination charges vs separate charges and transfers
 - Connected accounts (Express, Custom, Standard)
 - Payouts and balance management
 - Escrow patterns: holding funds until job completion
 - Dispute handling and refunds
- Stripe PaymentIntents documentation
 - "Funds on hold until confirmation" patterns
 - Capture vs authorization-only
 - Webhooks for payment lifecycle events
 - 3D Secure and SCA compliance
- Firebase Authentication documentation
 - Custom claims for roles (Client, Partner, Admin)
 - Multi-factor authentication
 - Social providers and email/password
 - Session management and token refresh
- Firebase Security Rules documentation
 - Multi-tenant patterns for Firestore
 - Role-based access control (RBAC)
 - Field-level security
 - Testing and debugging rules
- BigQuery, Cloud Logging, and Cloud Monitoring
 - Tenant-labeled usage tracking
 - Log sinks and structured logging
 - Custom metrics and dashboards
 - Alerting and anomaly detection
- n8n billing and usage tracking workflows
 - Scheduled workflows for nightly usage aggregation
 - Querying Firebase/Firestore for usage data
 - Calculating costs based on API usage (Groq, Inworld, ElevenLabs, etc.)
 - Generating invoices or updating user billing records
 - Sending usage alerts to Luc or admin dashboards

Key Research Questions for This Notebook

1. How to implement Fiverr-style escrow with Stripe Connect: when to capture vs transfer vs refund based on job lifecycle[1]
2. Mapping Client vs Partner vs Admin to Firebase Auth claims/roles and how that flows into routing (Gateways, Workspace pages, CircuitBox)[1][3]
3. How to track per-tenant usage across Groq, Inworld, ElevenLabs, Vertex AI, scrapers, and Docker containers, then expose that to Luc and to billing[1][3]

4. Whether to store transactional data (payments, contracts) strictly in Firestore vs also in Cloud SQL for analytics and Row Level Security[1][3]
5. Designing Firestore security rules that enforce tenant isolation and role-based access in line with the Hostinger + Firebase security notes[3]
6. How n8n can automate nightly billing calculations by aggregating usage data from multiple sources and updating user records[1][3]
7. Implementing rate limiting and quota enforcement per user tier (Free, Pro, Enterprise)[3]
8. How to surface usage data in Luc's UI for real-time cost estimation during conversations[1]

Prompts to Ask NotebookLM

- "Combine all Stripe / escrow references into a single, step-by-step payment flow for a Client hiring a Partner; from quote to release"[1]
- "Generate Firestore security rules that enforce tenant isolation and role-based access in line with the Hostinger + Firebase security notes"[3]
- "From Luc's spec and the PMO billing agent description, design a unified usage schema and nightly aggregation job"[1]
- "Design an n8n workflow that runs nightly to aggregate usage data from Groq, Inworld, Firebase, and GCP, then updates Firestore billing records"[1][3]
- "Propose a rate limiting strategy for API routes that varies by user tier and prevents abuse on shared Hostinger VPS"[3]

Index 6: n8n Workflow Automation Hub

Purpose: Centralize all n8n-specific patterns, workflows, and integrations for orchestrating agents, media pipelines, billing, and third-party services on Hostinger VPS[3].

Internal Documents to Upload

- **Locale-UI-and-Infrastructure-building.txt** – References to n8n as internal automation layer[1]
- **All previous BUILD DIRECTIVES** – Extract automation requirements from media pipeline, SME swipe, voice stack, and billing sections[2]
- **Hostinger strategy note** – n8n deployment on VPS alongside other Docker containers[3]
- **Any existing n8n workflow exports or screenshots** – If you have sample workflows already built[3]

External Web Documentation to Add

- n8n core documentation
 - Self-hosting with Docker on VPS
 - Security: authentication, HTTPS setup, and reverse proxy (Caddy/Nginx)
 - Workflow design patterns and best practices
 - Version control and backup for workflows
 - Environment variables and secrets management
- n8n node documentation
 - HTTP Request node (calling Agent Zero, OpenClaw, Groq, Inworld, ElevenLabs)
 - Webhook node (receiving triggers from Locale Next.js app)

- Firebase nodes (Firestore read/write, Auth)
- Google Cloud nodes (Cloud Tasks, Cloud Storage, BigQuery)
- Stripe node (payment processing, webhook handling)
- Schedule Trigger (cron jobs for billing, cleanup)
- Code node (JavaScript/Python for custom logic)
- Error handling and retry logic
- n8n workflow templates
 - Search n8n.io workflow library for relevant templates
 - Agent orchestration patterns
 - Billing and usage tracking workflows
 - Media generation pipelines
 - Notification and alerting workflows
- n8n scaling and performance
 - Queue mode for long-running workflows
 - Resource requirements and optimization
 - Monitoring execution logs and metrics
 - Troubleshooting common issues

Key Research Questions for This Notebook

1. How to deploy n8n as a secure Docker container on Hostinger VPS with proper authentication and HTTPS[3]
2. Network configuration: how n8n container communicates with Next.js, Agent Zero, OpenClaw, and external APIs[3]
3. Designing reusable n8n workflow templates for common Locale operations (agent orchestration, media pipeline, billing)[1][2][3]
4. How to trigger n8n workflows from Next.js API routes using webhooks or HTTP requests[1][3]
5. How to handle long-running workflows (video generation, batch processing) without blocking user requests[2][3]
6. Error handling strategy: when workflows fail, how to notify users and retry operations[3]
7. How to backup and version control n8n workflows alongside application code[3]
8. Resource allocation: how much CPU/RAM to allocate to n8n container on shared VPS[3]
9. Security: preventing unauthorized access to n8n workflows and protecting API keys stored in n8n[3]

Sample n8n Workflows to Build

- **Agent Orchestration Workflow**
 - Trigger: Webhook from Next.js when user submits a business architecture request
 - Steps: Call Agent Zero API → Parse response → Store result in Firestore → Trigger OpenClaw for follow-up tasks → Send completion notification
- **Media Pipeline Workflow**
 - Trigger: Scheduled or webhook when new SME persona created
 - Steps: Generate script with Gemini → Synthesize audio with Inworld/ElevenLabs → Create video with Key AI → Upload to Firebase Storage → Update Firestore with media URLs
- **Nightly Billing Workflow**

- Trigger: Schedule (daily at midnight)
- Steps: Query Firestore for usage data → Calculate costs → Update user billing records → Send usage alerts to admin dashboard
- **Voice Transcript Processing Workflow**
 - Trigger: Webhook when Groq Whisper returns transcript
 - Steps: Extract intent → Route to appropriate agent → Store conversation history → Update user context
- **Stripe Payment Workflow**
 - Trigger: Stripe webhook for payment events
 - Steps: Verify webhook signature → Update Firestore payment status → Trigger job start/completion logic → Send confirmation to user

Prompts to Ask NotebookLM

- "From all automation requirements in the BUILD DIRECTIVES, generate a prioritized list of n8n workflows to build for Locale"[1][2]
- "Design a Docker Compose configuration that includes n8n alongside Next.js, Agent Zero, and OpenClaw, with proper networking and security"[3]
- "Propose an error handling and retry strategy for n8n workflows that interact with external APIs like Groq, Inworld, and Stripe"[3]
- "Create a backup and version control plan for n8n workflows that integrates with GitHub and supports rollback"[3]
- "Design a resource monitoring dashboard for n8n that tracks workflow execution time, success/failure rates, and API costs"[3]

Getting Started with These Indexes

Step-by-Step Process

1. **Create 6 separate NotebookLM projects** - One for each index page above
2. **Upload internal documents first** - Start with your existing PDFs, markdown files, and specs
3. **Add external documentation** - Copy URLs or PDFs from the checklists above
4. **Ask initial research questions** - Use the prompts provided to get oriented
5. **Iterate and refine** - As you build, add new documents and ask follow-up questions
6. **Cross-reference between notebooks** - Some topics span multiple domains; use insights from one notebook to inform questions in another

Recommended Priority Order

1. Start with **Index 1 (Hostinger + Firebase + GCP)** - This is your foundation
2. Then **Index 3 (Voice Stack)** - Voice-first is your differentiator
3. Next **Index 2 (AI Agent Layer)** - Core functionality depends on agent orchestration
4. Then **Index 6 (n8n Workflows)** - Automation ties everything together
5. Then **Index 5 (Auth, Billing, Multi-Tenancy)** - Critical for production launch
6. Finally **Index 4 (Gateway, Verticals, Media)** - Polish and marketplace features

Maintenance and Updates

- Add new documents as you create implementation specs, API contracts, or architecture decision records
- Update external documentation when APIs change or new versions are released
- Periodically ask NotebookLM to "summarize recent changes" to stay current
- Use NotebookLM to onboard new team members by generating "quickstart guides" from each index

References

[1] Locale-UI-and-Infrastructure-building.txt – Comprehensive system architecture and component specifications

[2] BUILD DIRECTIVES collection – Media pipeline, SME swipe playground, and voice stack integration requirements

[3] Hostinger Strategy note – VPS deployment, Docker configuration, and multi-tenant security patterns